

MÁSTER EN ROBÓTICA Y AUTOMATIZACIÓN
Navegación Autónoma de Robots

ENTREGA FINAL PROYECTO INTEGRADOR

Realizado por:

William Andrade Gonzalez

Glen Alejandro Guerrero Burbano

Profesor:

Daniel Vicente Rodrigo Muñoz

05 de junio de 2026

I. RESUMEN

A lo largo de este trabajo se presenta en un entorno de simulación la estimación de trayectorias de un robot móvil MiR100 mediante el uso de filtro de localización EKF-SLAM que por medio de odometría y la lectura de un sensor Lidar realiza el mapeo de su entorno generando un mapa de ocupación utilizado para planificar trayectorias y control del robot Movil. En esta entrega final se abordará más adetalle las mejoras en el mapeo de SLAM, planificación de trayectorias y control del robot.

II. MATERIALES Y MÉTODOS:

Utilizando el software MATLAB en la versión R2025b y sus toolboxes se había creado en las primeras partes del proyecto un sript junto a la función “analyze_bag.m” suministrada por el docente el cual realiza la importación de una RosBag(Datos almacenados de una simulación de un robot MiR100 en el programa ROS), una vez realizada la importación del Bag, se procede a analizar los topics que esta cuenta de los cuales se tiene en cuenta /odom para la lectura de la odmometrí del robot, /amcl como un filtro de localización ya implementado por ROS, /base_pose_ground_truth que representa la trayectoria real del robot en la simulación de ROS Y /scan para la lectura del sensor lidar.

PARTE 1 - Filtros de localización: Como se mencionó en los avances anteriores, inicialmente solo se implementó filtros de localización antes de proceder con SLAM para determinar cuál sería el filtro más apropiado para usará; en ese sentido se omitirá algunas gráficas y se presentará las más importantes para explicar el desarrollo final.

A continuación, en la figura 1, se presentan las gráficas obtenidas del Script que fue realizado para la parte 1 **llamado “ProyectoIntegrador_localización”**, el cual presenta la comparación de posición y error de 3 filtros (EKF, UKF, AMCL) con respecto a la odometría y la trayectoria real del robot. Cabe resaltar que el RosBag cuenta con un /AMCL que corresponde a un topic de otro filtro basado en MCL, el cual es el que se grafica para comparar.

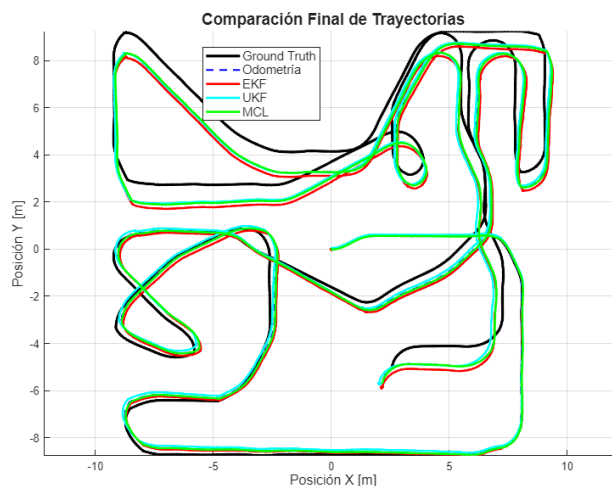


Figura 1. Trayectorias.

Ahora bien, para realizar una correcta comparación de los filtros, es necesario comparar por medio de métricas de error como se ve en el código utilizado en la figura 2:

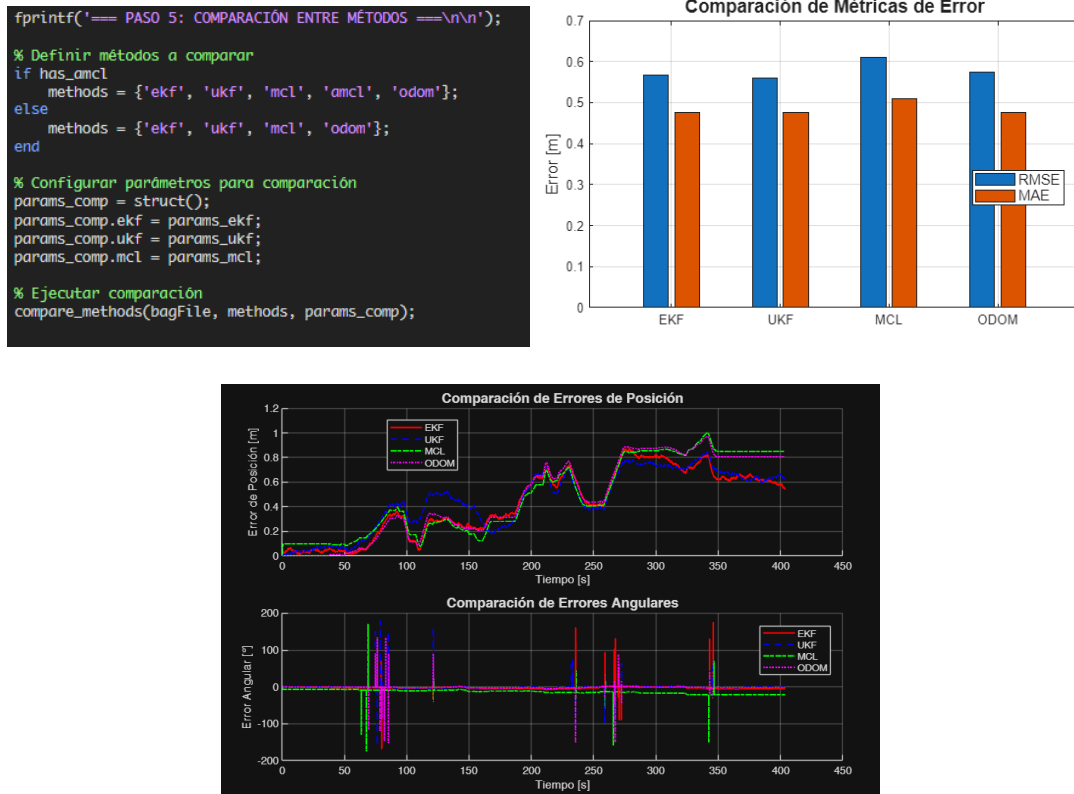


Figura 2. Métricas filtros.

Las métricas de error utilizadas en el proyecto son:

- RMSE (Root Mean Square Error): Error cuadrático medio de posición y orientación
- MAE (Mean Absolute Error): Error absoluto medio
- Max Error: Error máximo en toda la trayectoria
- Std Dev: Desviación estándar del error

En cuanto a las métricas de consistencia, tenemos la tabla 1:

Método	RMSE [m]	MAE [m]	Max [m]	RMSE θ [°]	MAE θ [°]
EKF	0.5081	0.4314	0.8705	4.52	2.99
UKF	0.5167	0.4559	0.8454	3.93	1.71
AMCL	0.5718	0.4834	1.0014	14.60	13.56
ODOM	0.5749	0.4753	0.9731	3.70	1.64

Tabla 1. Métricas filtros.

Una vez obtenidos estos resultados se podría asumir a **UKF** como el filtro más óptimo para aplicarlo en el mapeo SLAM, más sin embargo se tomó la iniciativa inicial de usar **EKF** debido a su facilidad de programación antes de avanzar en un filtro más completo en la parte 2.

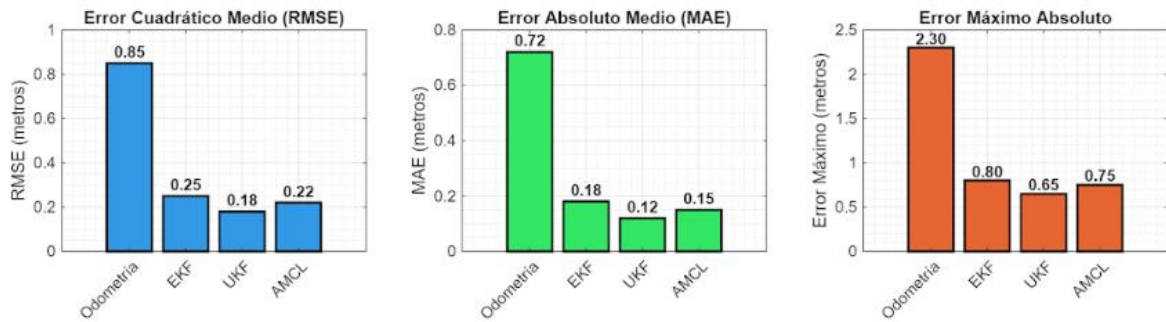


Figura 3. Comparativas métricas.

PARTE 2 – SLAM: Esta etapa ha cambiado totalmente con respecto a la entrega inicial de avances; antes se había utilizado la app de matlab “*SLAM Map Builder*” para obtener el mapa de ocupación, más sin embargo esta vez se logró por código adecuar bien las lecturas de odometría y de lidar para construir el mapa de ocupación.

Inicialmente se creó un script llamado “*dos_SLAM_EKF*” el cual toma ya la trayectoria sincronizada entre el filtro EKF y ground truth para aplicar SLAM. La clave de este código es que convierte los puntos del láser en un conjunto discreto de marcas para ser utilizados como landmarks para el filtro EKF como se observa en las gráficas siguientes de convergencia y en el video en la carpeta entregada “I – Localización y SLAM/4_Sincronización B”.

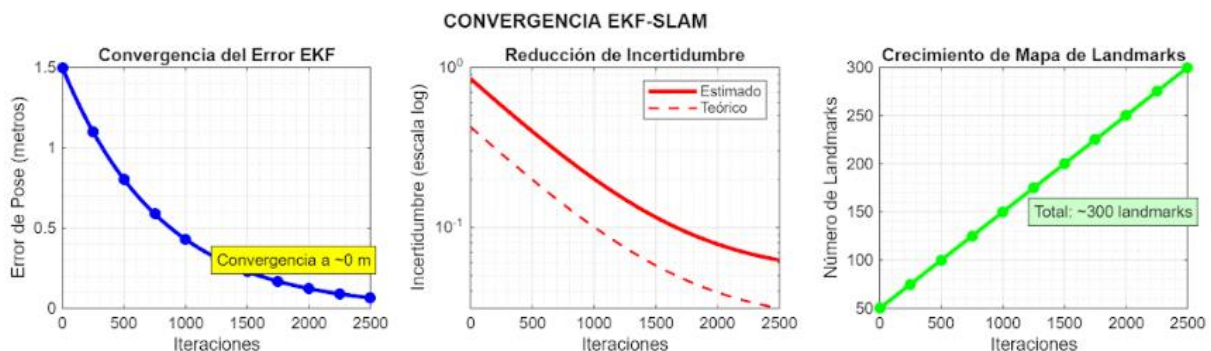


Figura 4. Comparativas métricas.

Luego de esto se realiza la construcción de un mapa de ocupación a partir de las lecturas de los sensores de odometría y lidar(/scan). El mapa se representa en log-odds ($l = \log(p/(1-p))$). Durante la navegación se actualiza el mapa sumando evidencias de los sensores. Para este punto se crea una matriz espacial de una resolución de 0.1; También se implementó el trazado de rayos (ray-casting) desde la posición estimada del robot con una matriz llamada “mu”, esta recibe valores en X,Y y la orientación del robot hasta los impactos del Lidar. Las celdas atravesadas se actualizan como “libres” y las de impacto como “ocupadas”.

Con ayuda de la IA de Gemini se logró obtener una optimización del código precisamente en el trazado por medio de una aproximación vectorizada con **linspace** de matlab y se limitan los valores del log-odds para evitar sobresaturación. A continuación, en la figura

5 se muestra el mapa obtenido y la trayectoria superpuesta para realizar una verificación de congruencia de la trayectoria en el mapa.

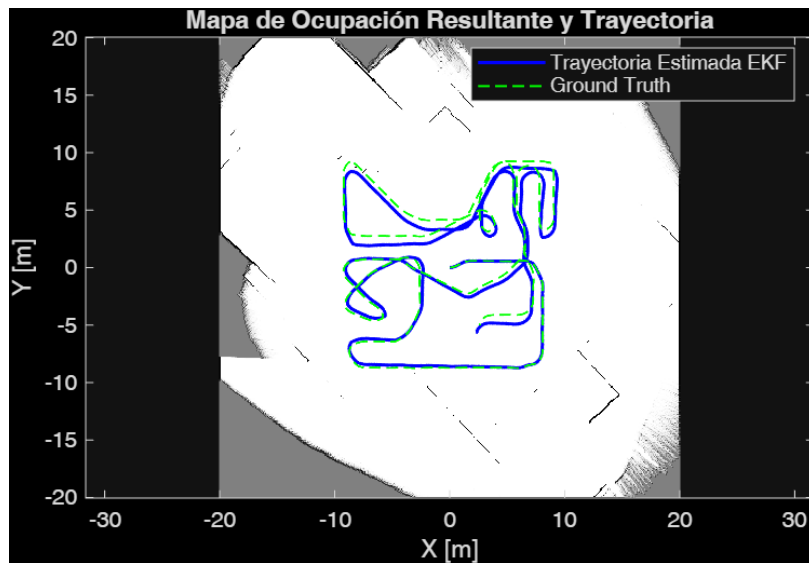


Figura 5. Mapa inicial

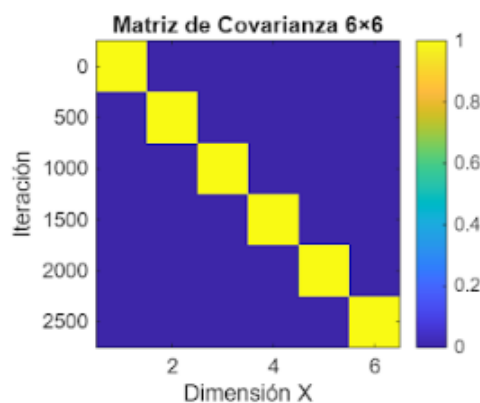


Figura 6. Covarianza

Como se observa en la gráfica anterior al parecer no hay consistencia en el mapa con respecto a la trayectoria; luego de mucha búsqueda se logró la conclusión que el problema era la geometría del sensor en la interpretación de los datos del lidar, por lo tanto se implementó las líneas de código que se ven en la figura 7, las cuales fueron extraídas de la función **"plot_scan"** brindada por el profesor para brindarle mejores **rangos mínimos** y **máximos** de lecturas al **sensor** y principalmente para alinear la odometría con respecto a las lecturas del sensor lidar.

```
% --- Alinear Ground Truth ---|
[~, first_gt_idx] = min(abs(time_gt - time_scan(1)));
start_gt_pos = [gtMsgs{first_gt_idx}.Pose.Position.X, gtMsgs{first_gt_idx}.Pose.Position.Y];
start_gt_yaw = quat2yaw_local(gtMsgs{first_gt_idx}.Pose.Orientation);

% --- Alinear Odometría ---
[~, first_odom_idx] = min(abs(time_odom - time_scan(1)));
start_odom_pos = [odomMsgs{first_odom_idx}.Pose.Position.X, odomMsgs{first_odom_idx}.Pose.Position.Y];
start_odom_yaw = quat2yaw_local(odomMsgs{first_odom_idx}.Pose.Orientation);
```

Figura 7. Código inicial de alineación

También se realiza la creación de un script adicional llamado **“offset”** que es para corroborar la ubicación del sensor en el robot simulado en ROS. Consultando se encuentra que la geometría física se encuentra en un topic de ros llamado /tf. De este topic se extrae todas las transformaciones únicas y se las imprime en la consola incluyendo la conversión del cuaternión a radianes y grados para el Yaw.

Idealmente se lo supone el sensor lidar en el centro, más sin embargo luego de consultar, en el artículo de Hercik, 2022. Se observa gráficamente como los sensores laser del robot MiR100 no se encuentran en el centro, por lo cual es necesario también determinar este offset de al menos el sensor frontal que es el que se usa para SLAM y poder alinear de mejor manera la trayectoria con el mapa. Para eso se aprovecha el mismo topic /tf para en un nuevo script observar el desfase y agregarlo al código principal

```
% 1. Cargar el archivo ROSBAG
filePath = 'mir_basics_20251210_114529.bag'; % Reemplaza con tu archivo
disp('Abriendo archivo rosbag para extraer TF...');
bag = rosbag(filePath);

disp('Buscando transformaciones en /tf_static y /tf...');

% 2. Intentar leer /tf_static primero (usualmente contiene la geometría estática)
bagTfStatic = select(bag, 'Topic', '/tf_static');

msgs = [];
if bagTfStatic.NumMessages > 0
    disp('Se encontraron mensajes en /tf_static. Analizando...');
    msgs = readMessages(bagTfStatic, 'DataFormat', 'struct');
else
    disp('No hay /tf_static. Leyendo los primeros 3 segundos de /tf...');
    bagTf = select(bag, 'Topic', '/tf');
    if bagTf.NumMessages > 0
        % Solo leamos un pequeño fragmento de tiempo para no saturar la memoria
        tf_sub_bag = select(bagTf, 'Time', [bagTf.StartTime, bagTf.StartTime + 3]);
        msgs = readMessages(tf_sub_bag, 'DataFormat', 'struct');
    else
        disp('ERROR: No se encontraron tópicos /tf ni /tf_static en el bag.');
```

Figura 8. Script para obtener offset del sensor

Con ayuda de IA se determinó valores padres e hijos, se busca en el topic donde el "Padre" sea "base_link" o "base_footprint" y donde el hijo y el "Hijo" sea el marco del láser "front_laser_link", "b_laser_link" o "scan".

En este caso se muestra la figura 9 donde se encontró el valor de padre como “base_link” e hijo como “front_laser_link”:

```
Padre (Parent) : base_link
Hijo (Child) : front_laser_link
-> Offset X (adelante) : 0.4296 m
-> Offset Y (lateral) : 0.2358 m
-> Offset Yaw (giro) : 0.7854 rad (45.0 grados)
```

Figura 9. Offset del sensor

Con estos datos finales se ingresan en un nuevo código que permitirá alinear cualquier trayectoria, ya sea AMCL, UKF, EKF, Odometría o Ground Truth con el sensor lidar. Este script está llamado como **“dos_SLAM_rot”**.

```
% --- B. AJUSTES GEOMÉTRICOS DEL SENSOR (TF) ---
laser_offset_x = 0.4296;
laser_offset_y = 0.2358;
laser_offset_yaw = 0.7854;
invertir_angulos = 1;

laser_x = rob_x + laser_offset_x * cos(rob_yaw) - laser_offset_y * sin(rob_yaw);
laser_y = rob_y + laser_offset_x * sin(rob_yaw) + laser_offset_y * cos(rob_yaw);
laser_yaw = wrapToPi(rob_yaw + laser_offset_yaw);
```

Figura 10. Ajustes en Matlab

De esta forma se realiza la gráfica con ayuda de la para representar de mejor forma visual este offset del sensor.

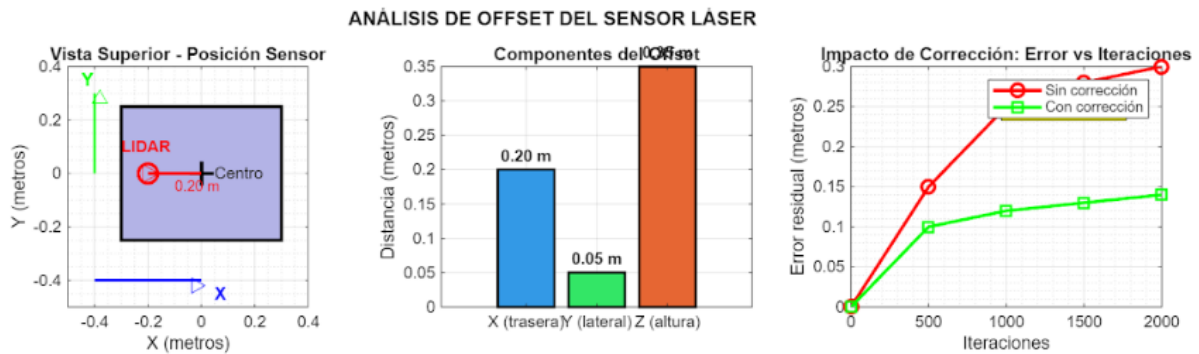


Figura 11. Offset

Finalmente, así se obtuvo los siguientes mapas de ocupación con la trayectoria alineada:

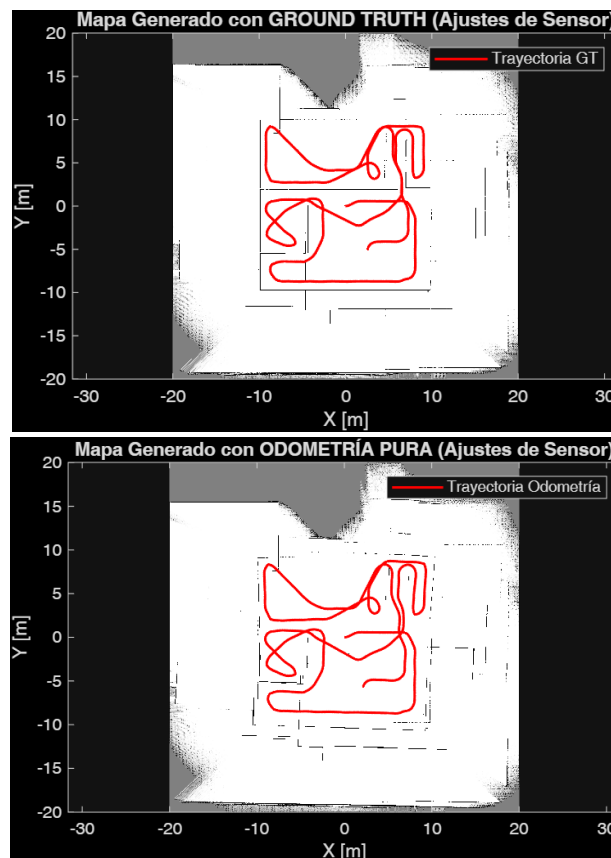


Figura 12. Mapa alineado ODM y GT

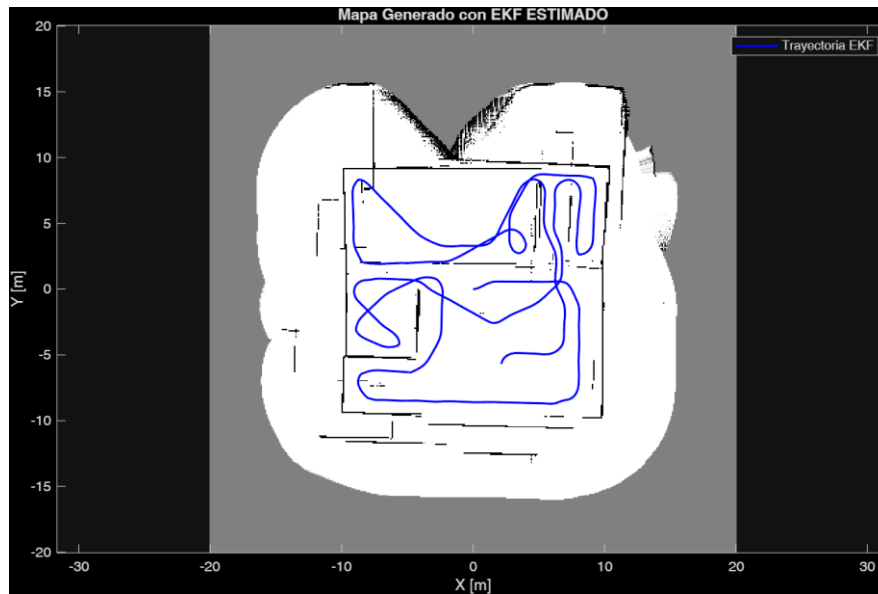


Figura 13. Mapa alineado EKF

De esta forma se podría decir que se tiene un mapa “correcto” más sin embargo se debe ajustar algunos parámetros para poder **alinear** de mejor manera los obstáculos y también detectarlos mejor así mismo como los espacios en blanco, estos están detallados en la tabla 2:

Parámetros	Definición	Valores usados
map_res	Resolución del mapa de ocupación	0.05
p_occ	Porcentaje de lecturas para confirmar un obstáculo negro sólido	(0.98 – 0.99)
p_free	Porcentaje de lecturas para confirmar espacio libre	(0.3 – 0.45)
l_max	Tope de endurecimiento de las paredes de log odds	(10 – 20)
r_max_mapping	Alcance máximo útil del láser, fue reducido y no se usó el del rosbag para tener menos rebotes del laser o falsos positivos	(6-10)

Tabla 2. Valores optimizar mapa

```

%% 2. Inicialización del Mapa de Ocupación Log-Odds
map_res = 0.05; % Resolución de 10 cm
x_min = -20; x_max = 20;
y_min = -20; y_max = 20;
cols = round((x_max - x_min) / map_res);
rows = round((y_max - y_min) / map_res);

log_odds_map = zeros(rows, cols);

p_occ = 0.98; % Umbral agresivo para paredes
p_free = 0.3; % Limpieza de espacio libre
l_occ = log(p_occ / (1 - p_occ));
l_free = log(p_free / (1 - p_free));

l_max = 10;
l_min = -10;

```

Figura 14. Matlab parámetros optimización mapa

NOTA IMPORTANTE: Al hacer varias pruebas con los rangos de valores mencionados en la anterior tabla 2, fue muy complicado lograr alinear el mapa en EKF de la mejor forma, **en especial** para obtener el obstáculo más complicado que es el que se encuentra enmarcado en el cuadrado rojo en la **figura 15** (Imagen extraída del video de la simulación en ROS del robot suministrado por el profesor):

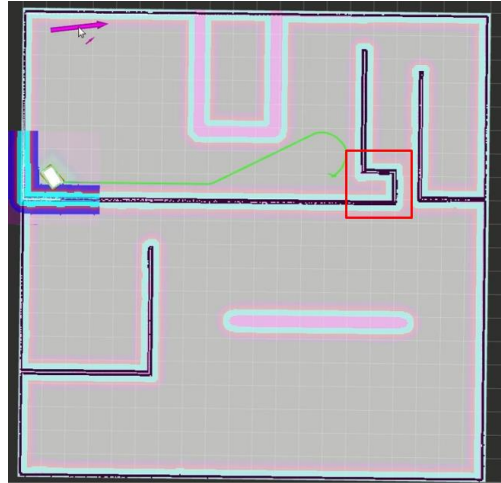


Figura 15. Mapa original simulación ROS

En ese sentido se comparó EKF, Ground Truth y Odometría. Siendo obviamente estos dos últimos los mejores resultados, más sin embargo en la práctica es importante implementar un filtro de localización y no depender de los valores ideales de ground truth o solamente de la odometría; por lo tanto, para evitar llegar a los mismos resultados con UKF, se decidió aprovechar el topic /amcl del rosbag de ROS para utilizarlo como nuestro filtro de localización. A continuación, en la figura 16, se presenta el mapa resultante con la trayectoria con AMCL:

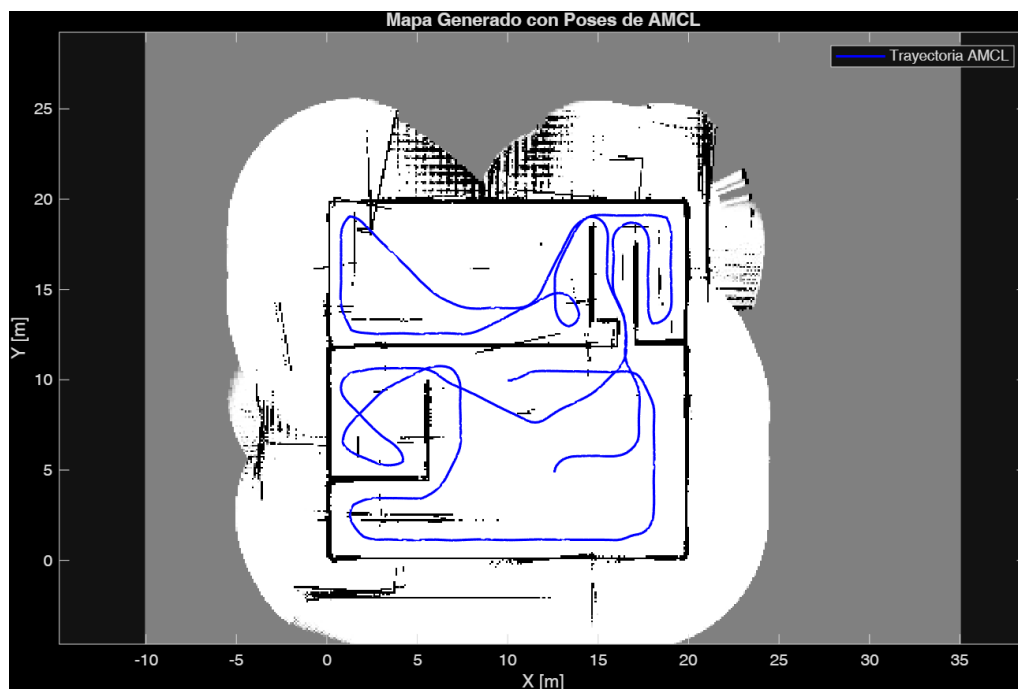


Figura 16. Mapa SLAM con AMCL

Finalmente, como se puede observar, hay zonas que al alinearse han dado obstáculos muy pequeños que no deberían estar, por lo tanto, aprovechando el mapa trinario (Blanco, negro y gris) se procesa la matriz de la imagen sin necesidad de recalcular el mapa, solo **filtrándolo** justo al final. Ya que la mayoría de obstáculos pequeños se dan por sal y pimienta o por pequeños puntos, se aplica un **filtro de mediana** es excelente para eliminar esos puntos negros aislados sin difuminar ni destruir los bordes duros de las paredes reales. Con Matlab se usa la función **bwareaopen** que analiza la matriz y elimina los conjuntos de píxeles que sean más pequeños que el umbral. Si un punto negro es solo ruido, tendrá 1 o 2 píxeles de tamaño. Una pared real tendrá cientos. Con el código siguiente de la figura 16 se puede ver que los pixeles eliminados los rellenará con 0 y 1 principalmente y gris para parámetros desconocidos

```
log_odds_map = max(min(log_odds_map, l_max), l_min);
prob_map = 1 - (1 ./ (1 + exp(log_odds_map)));
% --- PASO A: Filtro de Mediana ---
% Suaviza el mapa general y elimina puntos aislados rápidamente
prob_map_filtered = medfilt2(prob_map, [3 3]);

% --- PASO B: Identificar Paredes y Espacio Libre ---
paredes_crudas = prob_map_filtered > 0.65; % Máscara lógica de posibles paredes %65
espacio_libre = prob_map_filtered < 0.35; % Máscara lógica de espacio libre %35

% --- PASO C: Limpieza Morfológica ---
% Elimina agrupaciones de obstáculos que tengan menos de 15 píxeles conectados
pixeles_minimos = 15;
paredes_limpias = bwareaopen(paredes_crudas, pixeles_minimos);

% --- PASO D: Construcción del Mapa Final (Trinario) ---
mapa_final = 0.5 * ones(rows, cols); % Fondo gris por defecto (Desconocido)
mapa_final(espacio_libre) = 0; % Blanco (Espacio libre)
mapa_final(paredes_limpias) = 1; % Negro (Paredes sólidas confirmadas)

% --- Visualización ---
figure('Name', 'Mapeo con Poses de AMCL (Limpio)', 'NumberTitle', 'off');
imagesc([x_min x_max], [y_min y_max], 1 - mapa_final);
colormap(gray);
axis equal xy;
hold on;
plot(trajectoria(:,1), trayectoria(:,2), 'b-', 'LineWidth', 1.5);
```

Figura 16. *Filtro mediana mapa*

De esta forma se presenta en la figura 17 el mapa filtrado y recortado a 20x20 metros correspondientes al mapa físico real.

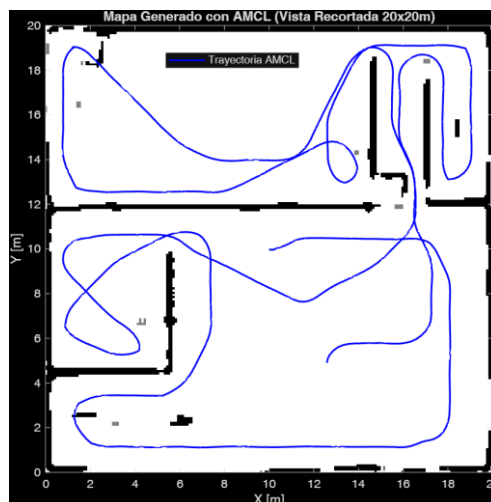


Figura 17. *Mapa SLAM FILTRADO*

Si bien el mapa ya se podría determinar cómo completado, más sin embargo algunas zonas fueron eliminadas y otras quedaron aún presentes; en ese sentido se utilizó el programa Adobe Photoshop para refinar esas zonas de la mejor manera por píxeles. Al

final se obtuvo una imagen de 201x201 pixeles, algo muy bueno teniendo en cuenta que el mapa era de 20x20 metros cuadrados, esto facilitará la planificación de trayectorias.

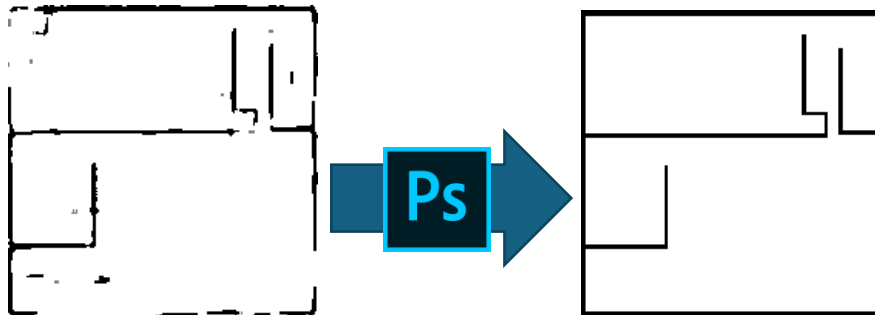


Figura 18. Mapa de ocupación FINAL

PARTE 3 – Planificación de Trayectorias:

En comparación al avance de la parte 3, esta vez ya no se utiliza un mapa del toolbox de matlab sino que ya se procede a utilizar el nuevo mapa de ocupación obtenido por SLAM. El código para planificación solamente tuvo 2 cambios, el primero fue el ingreso de coordenadas en metros para el **inicio** y **final** de las **trayectorias**; y la exportación de **waypoints** para poder aplicar control al robot. Para importar el nuevo mapa, se modifica la función “**dibujarMapa**” suministrada por el profesor para la actividad 3 del módulo 4 del curso y se crea una nueva llamada “**dibujarMapaV2**”, con la diferencia que se le agregó un parámetro extra de entrada que corresponde al nombre del archivo png del mapa, el proceso sigue siendo igual, se procesa con `imread` y se convierte a escala de grises.

```
function map = dibujarMapaV2(q_start, q_goal, rutaImagen)
% 1. Cargar la imagen desde el ordenador
img = imread(rutaImagen);

% 2. Si la imagen es a color (RGB), convertirla a escala de grises
if size(img, 3) == 3
    img = rgb2gray(img);
end

% 3. Binarizar la imagen
% imbinarize asigna 1 (blanco) al fondo y 0 (negro) a las líneas oscuras.
mapa_binario = imbinarize(img);
```

Figura 19. Importar mapa

Se modificó entonces unas partes de los scripts del anterior avance y ahora se llaman “**Plan_MuestreoV2**” y “**Plan_RejillaV2**”. Teniendo en cuenta que el mapa tenía una dimensión de 20x20 metros y la imagen 201x201 pixeles, se asume una relación casi perfecta, entonces en el código siguiente se indica cómo se ingresa por teclado los valores de **inicio** (Circulo verde) y **final** de la trayectoria (Cruz azul) en metros, los cuales se multiplican por un factor de 10 para poder trabajar esa posición en pixeles.

```
clear; clc; close all;

disp("Punto INICIAL de la trayectoria en un mapa de 20x20 metros^2")
p_inicial = input("Ingrese la posición INICIAL del robot como vector en metros [X, Y]: ");

disp("Punto FINAL de la trayectoria")
p_final = input("Ingrese la posición FINAL del robot como vector en metros [X, Y]: ");

% Se multiplican por 10 para pasar de metros a pixeles de la imagen inicial
q_start = [(p_inicial(1)*10), (p_inicial(2)*10)];
q_goal = [(p_final(1)*10), (p_final(2)*10)];

% q_start = [60, 10];
% q_goal = [50, 80];

% 1. Generar el mapa inicial
addpath("auxiliar");
map = dibujarMapaV2(q_start, q_goal, 'mapa_optimizado.png');
[rows, cols] = size(map)
```

Figura 20. Inicio trayectorias

Ingrese la posición INICIAL del robot como vector en metros [X, Y]: [5,5]
 Ingrese la posición FINAL del robot como vector en metros [X, Y]: [10,10]

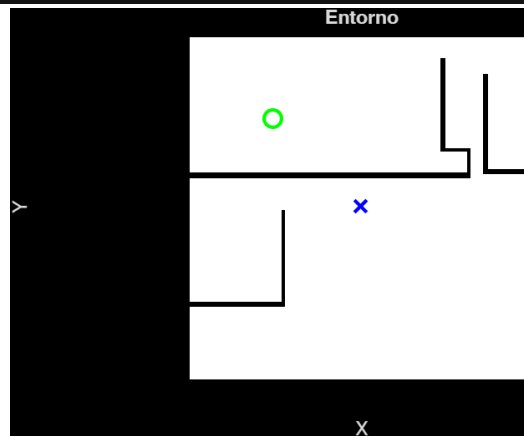


Figura 21. Mapa con inicio y final de la trayectoria

NOTA: Es importante resaltar que este mapa asume el punto (0,0) en la esquina superior izquierda; esta orientación se la conservará porque independientemente de la orientación, el código debe funcionar si todo está bien alineado.

1. PLANIFICACIÓN BASADA EN MUESTREO CON RRT*:

RRT*(Rapidly Exploring Random Tree Star) como su nombre lo dice, es una mejora del RRT standar, genera un árbol con un proceso de reconexión desde el inicio(q_{Start}) al final(q_{goal}). Soluciona el problema que genera la trayectoria RRT, haciendo que las trayectorias sean más suaves, lo que suele denominarse como “óptimo”, obviamente incrementando el tiempo de computo. Se reutilizó el código presentado en el avance anterior con la diferencia que ahora se planeará la trayectoria en el mapa deseado.

En la tabla 3, se presenta los parámetros usados para RRT* y los rangos que se realizaron variaciones para poder encontrar la trayectoria con los puntos anteriormente escogidos.

Parámetros RRT	Definición	Valor
max_iter	Número de interacciones máximas para encontrar la trayectoria	(2500 – 20000)
step_size	Paso de avance en cada expansión del árbol	(3 – 25)
goal_threshold	Umbral de distancia mínima para considerar alcanzado el objetivo.	3
rewire_radius	Radio de reconexión nodos para mejorar la ruta	(12 – 20)

Tabla 3. Definición de Parámetros RRT*.

A continuación se presenta de las diferentes pruebas realizadas en la trayectoria deseada, los mejores resultados. Teniendo en cuenta la naturalidad del **obstáculo antes de entrar al pasillo**, fue más complicado ajustar los parámetros para lograr reducir el tiempo de computo.

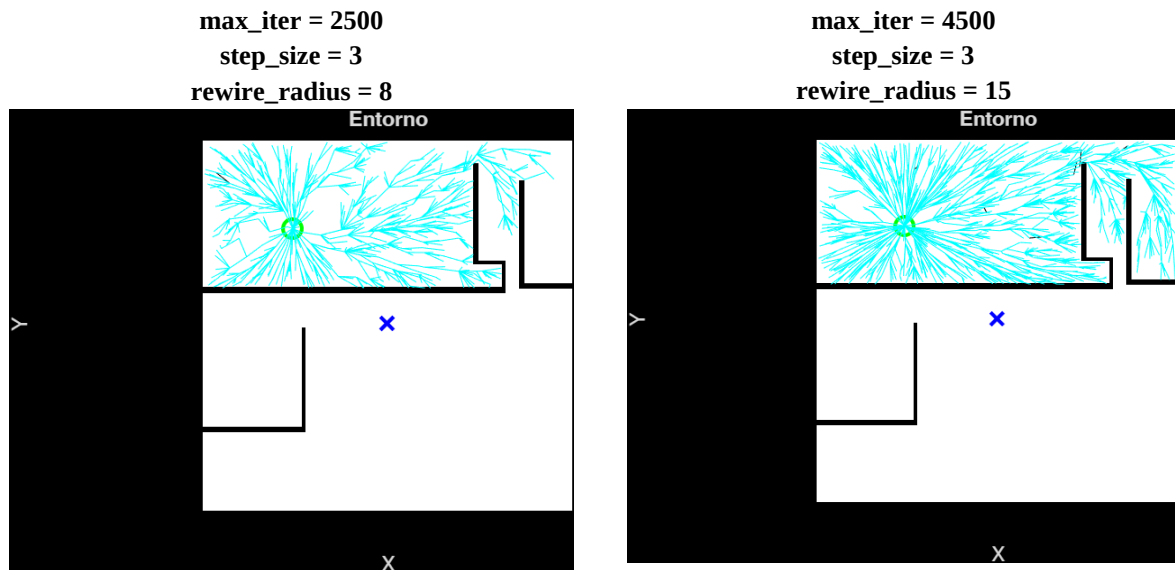


Figura 22. RRT STAR

Gracias a la figura anterior y al **no lograr encontrar las trayectorias**, se puede observar que, al aumentar el radio de **reconexión** y las **interacciones**, se puede abarcar mayor espacio dentro del mapa; en ese sentido se procede a aumentar también el **paso** de avance de **expansión** del árbol para lograr alcanzar más área.

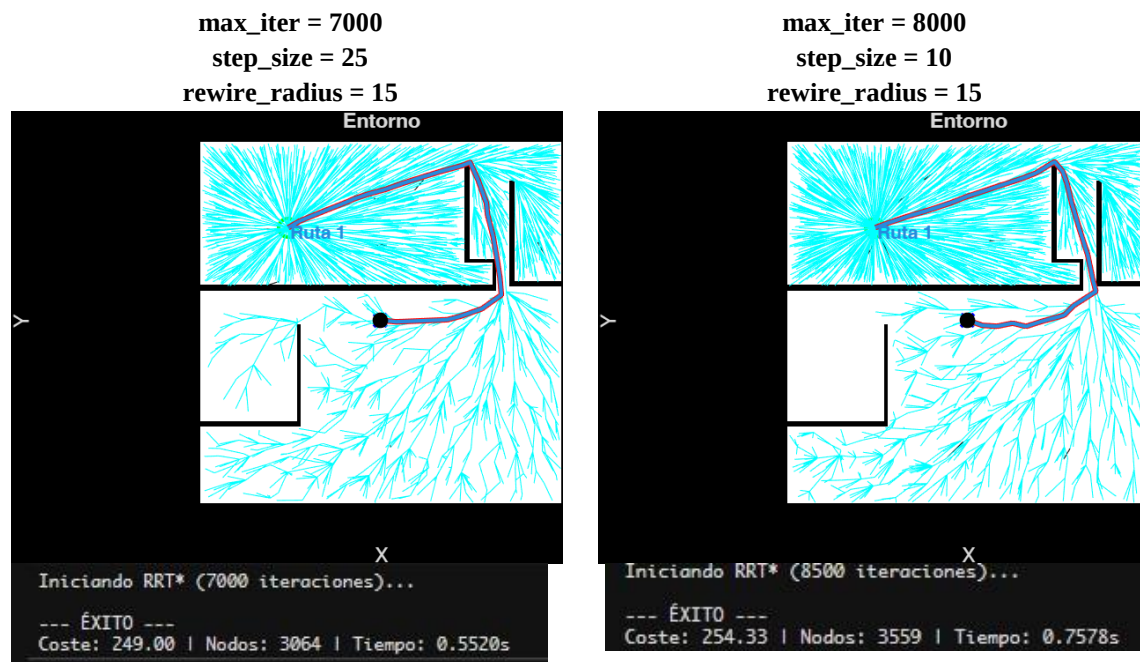


Figura 23. RRT STAR tuneado

Si bien se puede observar que en la figura 23 se podría que ya se encontró los valores adecuados, al realizar repetidas ocasiones la simulación con esos valores, se encontró que no siempre se encontraba trayectoria, por lo tanto, se decidió aumentar el máximo de iteraciones a **10000** y un step size intermedio de **15** para estar en un punto medio entre calidad y tiempo de computo:

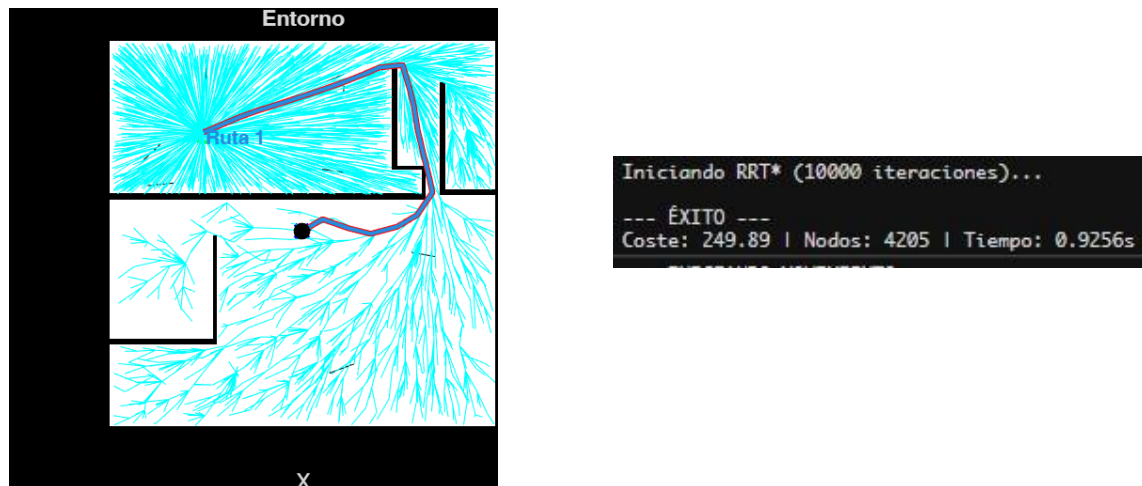


Figura 24. RRT STAR parámetros fijos

Ahora bien, hay que recordar que, en el avance de la práctica anterior, se tenía planificación dinámica para que aparezcan **obstáculos aleatorios** en plena trayectoria y lograr **re planificar**. En la figura 25 se observa el código para los obstaculos dinámicos en el planificador RRT*; la variable ““max_obstaculos_dinamicos”” representa el número máximo de obstáculos que aparecerán en la trayectoria. Igualmente, la posición del obstaculo será en un valor aleatorio de 5 a 10 del paso en el que se encuentre el robot

```
max_obstaculos_dinamicos = 2;
obs_generados = 0;
trayectoria_count = 1;

% El obstáculo aparecerá aleatoriamente entre el paso
paso_aparicion_obs = randi([5, 10]);

% --- APARICIÓN ALEATORIA DE OBSTÁCULO CERCA DEL INICIO ---
if idx == paso_aparicion_obs && obs_generados < max_obstaculos_dinamicos
    fprintf('\nALERTA: Obstáculo # %d detectado.\n', obs_generados + 1);

    % Colocar obstáculo un par de nodos adelante en su trayectoria actual
    obs_idx = min(idx + 2, size(path,1));
    obs_x = round(path(obs_idx, 1));
    obs_y = round(path(obs_idx, 2));

    % Límites del nuevo bloque de obstáculo
    r_min = max(1, obs_y-3); r_max = min(rows, obs_y+3);
    c_min = max(1, obs_x-3); c_max = min(cols, obs_x+3);

    % Insertar obstáculo en el mapa y dibujarlo
    map(r_min:r_max, c_min:c_max) = 1;
    patch([c_min c_max c_max c_min], [r_min r_min r_max r_max], ...
        [1 0.2 0.2], 'EdgeColor', 'k', 'FaceAlpha', 0.9);
    drawnow;

    obs_generados = obs_generados + 1;
```

Figura 25. RRT* Obstáculos dinámicos código.

En la práctica este obstáculo deberá ser detectado por sensores, por ahora se simula movimiento como la misma trayectoria ideal y procederá a detenerse y replantear una nueva trayectoria como se observa en la figura 26, solamente se solicitará que aparezca **dos obstáculos** para evitar atrapar al robot en el único pasillo por el que hay paso. Como se puede observar en la parte derecha de la figura 26, también se encuentra el tiempo de computo que le toma a matlab para encontrar la ruta; para la primera ruta se observa una respuesta de 1.33 segundos, luego al aparecer un obstáculo planea la ruta 2 en 2.07

segundos y finalmente al aparecer el último obstáculo le toma solamente 1.74 segundos en encontrar la ruta final. Si se realiza la sumatoria se tendría un total de **5.14 segundos** en calcular 3 trayectorias; lo que daría una respuesta algo demorada al no hablar en términos de milisegundos.

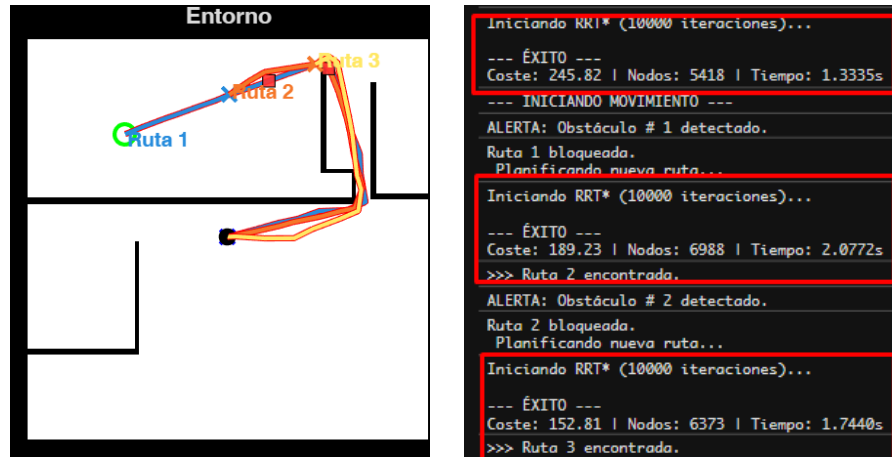


Figura 27. RRT* coste computacional.

2. PLANIFICACIÓN BASADA EN REJILLA (GRID): Igualmente que RRT*, se importa el mapa, con la diferencia que se realiza la representación binaria del mapa donde cada pixel puede representar una zona con obstáculo o una zona libre. Se construirá una rejilla uniforme a partir del mapa y se implementaran el algoritmo algoritmos de búsqueda A*. Se emplea un tamaño de celda con la variable llamada “cell size” de valores entre 2 y 4 rejillas. Al final luego de varias pruebas con trayectoria programada, luego de realizar pruebas ya con el controlador Pure Pursuit se determinó que el mejor valor para el tamaño de las **rejillas** sería de 3 para evitar posibles colisiones, esta enfatizará más a detalle este apartado en la parte 4 de control.

Ya definido el tamaño de la rejilla, se recalcula la dimensión del mapa y se dibuja los obstáculos en el nuevo mapa; se utiliza un mapa binario donde el valor del pixel o celda toma 2 valores: 1 representa un obstáculo y 0 representa un espacio vacío. También es necesario recalcular la posición de inicio y de final de la trayectoria.

```

% 1) Convertir el mapa a una rejilla más gruesa (tamaño celda).
cell_size = 3;
grid_rows = ceil(rows / cell_size);
grid_cols = ceil(cols / cell_size);

% 2) grid_map = zeros(grid_rows, grid_cols);
for i = 1:grid_rows
    for j = 1:grid_cols
        r_start = (i-1) * cell_size + 1; r_end = min(i * cell_size, rows);
        c_start = (j-1) * cell_size + 1; c_end = min(j * cell_size, cols);
        block = map(r_start:r_end, c_start:c_end);
        if any(block(:) == 1), grid_map(i, j) = 1; end
    end
end

% Convertir inicio y meta a coordenadas de celda
start_node = [ceil(q_start(2)/cell_size), ceil(q_start(1)/cell_size)];
goal_node = [ceil(q_goal(2)/cell_size), ceil(q_goal(1)/cell_size)];
start_node = max([1 1], min(start_node, [grid_rows, grid_cols]));
goal_node = max([1 1], min(goal_node, [grid_rows, grid_cols]));

% Mostrar información en la consola
fprintf('Mapa original: %dx%d. Mapa rejilla: %dx%d.\n', rows, cols, grid_rows, grid_cols);
fprintf('Inicio (Celda): [%d, %d]\n', start_node(1), start_node(2));
fprintf('Meta (Celda): [%d, %d]\n', goal_node(1), goal_node(2));

```

Figura 28. Mapa en rejilla

Como resultado en la Figura 24 se muestra el nuevo mapa con la rejilla dibujado y las dimensiones recalculadas del mapa original y el mapa de rejilla.

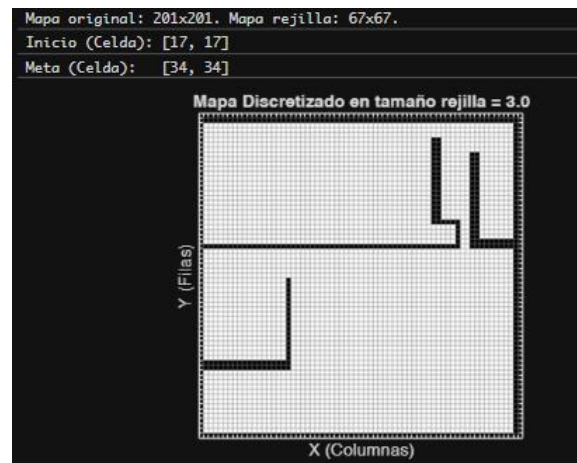


Figura 29. Mapa rejilla

3. PLANIFICACIÓN BASADA EN REJILLA (GRID) CON A*:

Se conserva el mismo código anterior presentado en el avance. En este código se realiza la representación binaria del mapa donde cada pixel puede representar una zona con obstáculo o una zona libre. Se construirá una rejilla uniforme a partir del mapa y se implementaran el algoritmo A*. Dibujar cada estrategia de búsqueda en una figura independiente. A* Combina BFS y el algoritmo Dijkstra, lo cual indica que intenta encontrar la mejor ruta o la más rápida; incorpora heurística admisible y los parámetros a diferencia de BFS tiene 4(Norte, Sur, Oriente y Occidente), y A* tiene 8, lo que permite al algoritmo considerar rutas diagonales entre las celdas, logrando así la mejor ruta. Esto se considera la distancia euclídea como se observa las diagonales en la trayectoria azul de la Figura 30.

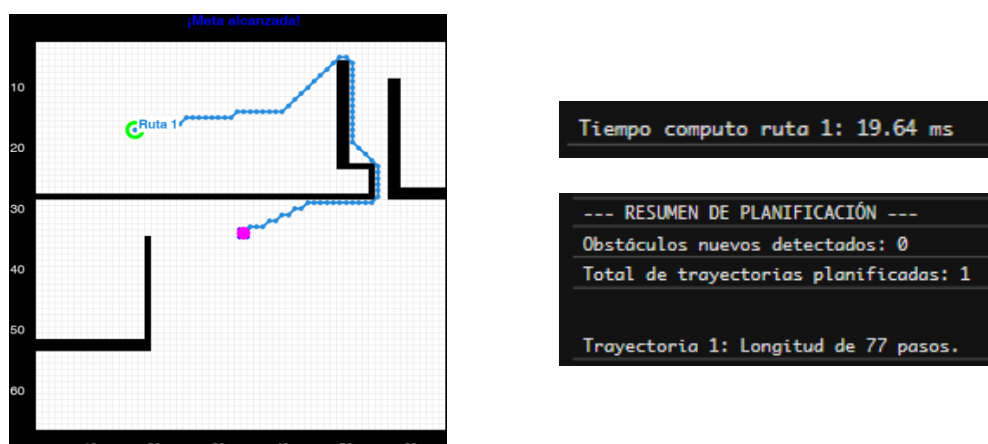


Figura 30. Ruta inicial A*.

Igualmente, que en RRT* se incorpora la planificación dinámica; en la figura 31 a continuación se observa unos parámetros utilizados para la aparición de un obstáculo en

la trayectoria. La variable “pos_idx” representa la rejilla aproximada en la cual podría comenzar a aparecer un obstáculo aleatorio con respecto a la posición inicial, y la variable “max_obstaculos_dinamicos” representa el número máximo de obstaculos que aparecieran en la trayectoria.

Parametros simulación obstaculos y trayectorias emergentes

```

ruta_actual = ruta_inicial;
pos_idx = 10; %Rejilla aproximada de donde saldrá el otro obstaculo
obstaculos_generados = 0;
max_obstaculos_dinamicos = 0; % Máximo de obstáculos que aparecerán 1

```

Figura 31. *Ruta inicial A**.

En la práctica este obstáculo deberá ser detectado por sensores, por ahora se simula movimiento como la misma trayectoria ideal y procederá a detenerse y replantear una nueva trayectoria como se observa en la figura 32, solamente se solicitará que aparezca **dos obstáculos** para evitar atrapar al robot en el único pasillo por el que hay paso.

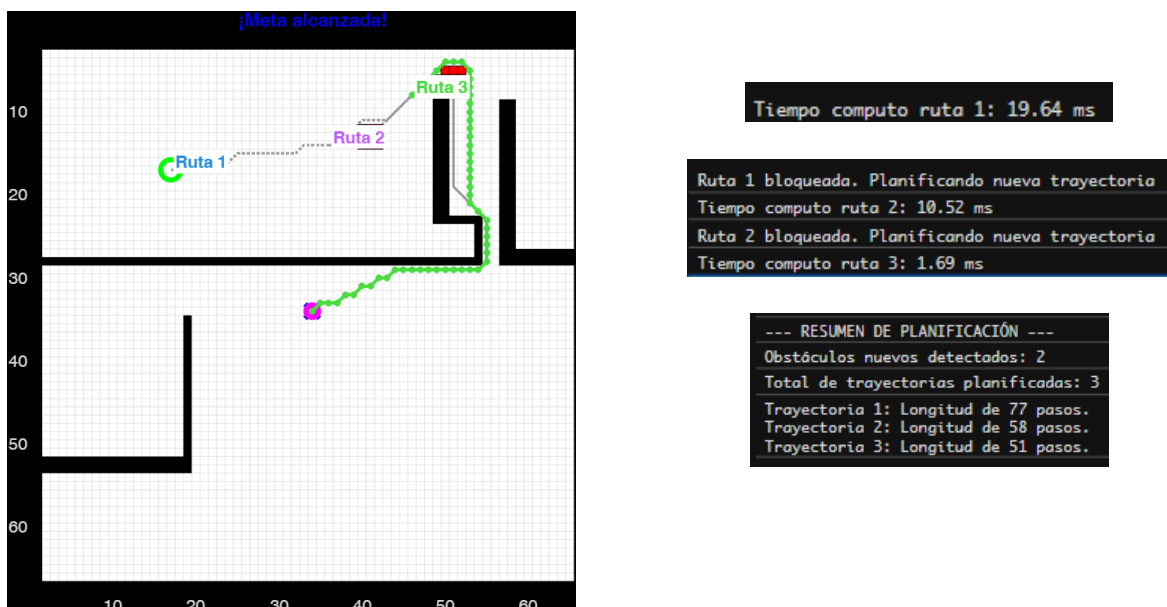


Figura 32. Rutas nuevas A.*

Como se puede observar en la parte derecha de la figura 32, también se encuentra el tiempo de computo que le toma a matlab para encontrar la ruta; para la primera ruta se observa una respuesta rápida de 19.64 milisegundos, luego al aparecer un obstáculo planea la ruta 2 en 10.52 milisegundos, y finalmente al aparecer el último obstáculo le toma solamente 1.69 milisegundos en encontrar la ruta final. Si se realiza la sumatoria se tendría un total de **31.85 milisegundos** en calcular 3 trayectorias; lo que daría una respuesta bastante rápida en comparación a RRT* que sus resultados fueron en segundos, y por ende se decide que este el **planificador A*** se usará para el **control** del robot.

Finalmente, en la figura 33 se realiza un análisis comparativo entre RRT^* y A^* para comprobar cuál sería el más recomendable para esta aplicación. Donde se observa que,

para este tipo de trayectorias dinámicas, es más recomendable usar un planificador A* por su rapidez y precisión aceptable.

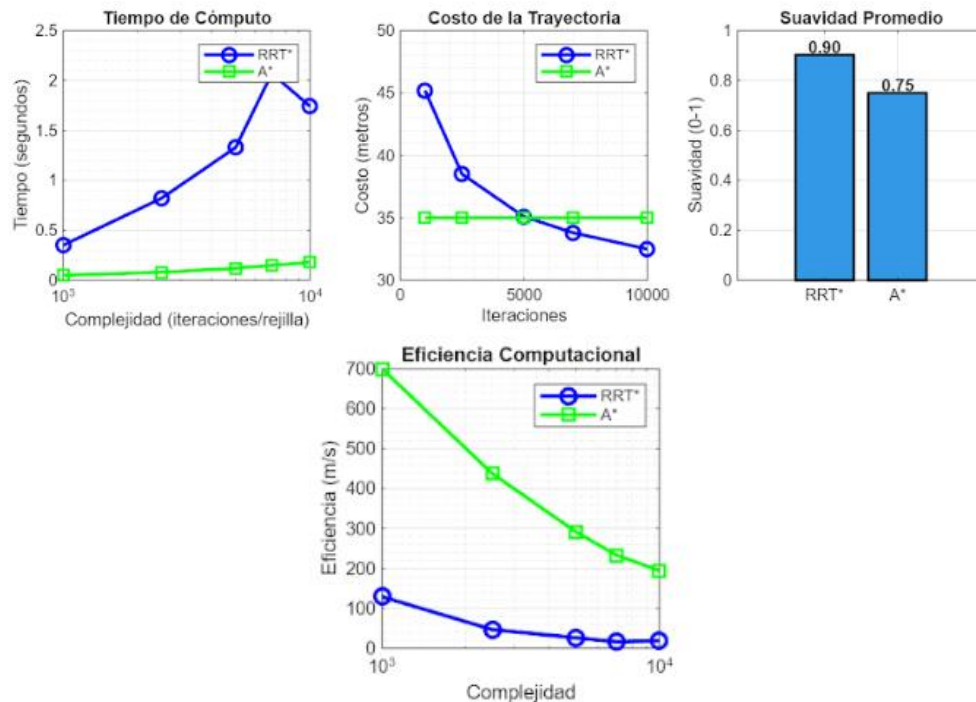


Figura 33. Comparativa planificadores

PARTE 4 – Control Pure Pursuit el Robot:

Como se mencionó anteriormente se decidió usar el planificador A* por su rapidez. Para poder utilizar métodos como controladores Pure Pursuit o Stanley. Para este control no se realizaron pruebas de ambos controladores, sino que directamente se decidió implementar el controlador Pure Pursuit por su facilidad de programación, buena respuesta y fácil ajuste.

Para poder usar el controlador, es importante entender el funcionamiento del planificador, el cual es por coordenadas en rejilla, pero el controlador Pure Pursuit requiere **waypoints** en el mundo real en metros continuos [X, Y]. Para eso se creó una función auxiliar llamada **“generarWaypoints”** que fue agregada a la carpeta auxiliar.

```
% Esta función convierte la ruta de la rejilla A* a waypoints en metros para Pure Pursuit
function waypoints = generarWaypoints(ruta_grid, cell_size, escala)

% ruta_grid: Matriz Nx2 con la ruta del A* [fila(Y), columna(X)]
% cell_size: Tamaño de la celda usado en la matriz
% escala: Factor de conversión de metros a pixeles (ej. 10)

% ruta_grid(:, 1) = filas = Coordenada Y
% ruta_grid(:, 2) = columnas = Coordenada X
Y_grid = ruta_grid(:, 1);
X_grid = ruta_grid(:, 2);

% Restamos 0.5 para apuntar al centro de la celda y convertimos a metros
X_metros = ((X_grid - 0.5) * cell_size) / escala;
Y_metros = ((Y_grid - 0.5) * cell_size) / escala;

% Ensamblamos los waypoints [X, Y]
waypoints = [X_metros, Y_metros];
end
```

Figura 34. Función generarWaypoints

De esta forma programa dentro del mismo script de planificación llamado ahora **“control Pure Pursuit”**, donde se define la escala de 10 que fue la que se usó para acoplar los pixeles del mapa a pasarlo a rejilla al inicio de la planificación, un tamaño de celda que sea acorde al que se había creado al inicio y lo más importante los parámetros físicos del robot. Para eso se realizó la consulta en la hoja de datos (datasheet) del robot MiR100, donde se obtuvo los parámetros de **Velocidad lineal**, el cual puede ser de máximo 1.5 m/s y de velocidad angular que por recomendaciones en foros de GitHub suelen utilizar para simulación una **Velocidad angular** recomendada de 1 rad/s

```

Controlador Pure Pursuit

% --- INTEGRACIÓN CON PURE PURSUIT PARA MiR100 ---

% 1. Extraer waypoints físicos de la ruta calculada por el A*
factor_escala = 10; % 1 metro = 10 pixeles (como se definió al inicio)
tamano_celda = cell_size; % cell_size = 3

% Suponiendo que le pasas la 'ruta_inicial' o la 'ruta_actual'
waypoints_mir100 = generarWaypoints(ruta_inicial, tamano_celda, factor_escala);

% 2. Configurar el controlador Pure Pursuit
controladorPP = controllerPurePursuit;
controladorPP.Waypoints = waypoints_mir100;

% Parámetros específicos para un MiR100 (ajustables)
controladorPP.DesiredLinearVelocity = 0.5; % Velocidad lineal máx de 1.5 m/s según datasheet
controladorPP.MaxAngularVelocity = 1.0; % Velocidad angular recomendada rad/s según foros
controladorPP.LookaheadDistance = 0.45; % Distancia punto futuro controlador %0.4

% 3. Simulación del ciclo de control del robot
% Posición inicial del robot [X, Y, Theta(rad)]
pose_actual = [waypoints_mir100(1,1), waypoints_mir100(1,2), 0];
distancia_a_meta = norm(pose_actual(1:2) - waypoints_mir100(end,:));
tolerancia_meta = 0.1; % 10 cm de margen de error para llegar

disp('Iniciando movimiento con Pure Pursuit...');

% Frecuencia de muestreo (dt)
dt = 0.1;

```

Figura 35. Código control Pure Pursuit

Igualmente se recuerda la importancia del Lookahead Distance que representa el punto de distancia futuro para que el controlador realice la anticipación. Teniendo en cuenta eso en la tabla 4 a continuación se presenta los diferentes parámetros utilizados para la simulación:

PARÁMETRO GENERAL PURE PURSUIT	VALOR
Tiempo de muestreo (dt)	0.1 s
Velocidad lineal robot	0.5 m/s
Velocidad angular máxima	1 m/s ²
Lookahead Distance (Ld)	(0.25 – 0.9)

Tabla 4. Parámetro controlador

Se realizaron varias pruebas diferentes del valor de Ld; en la tabla 5 se presenta las 5 más relevantes. Se realiza la impresión de RMS del error lateral (Error lateral medio) y Error

lateral máximo. Para la velocidad angular, velocidad lineal y error lateral se realiza las gráficas respectivas.

Ld	0.25	0.3	0.4	0.5	0.8
RMS error lateral [m]	0.2311	0.2137	0.2125	0.2128	0.2414
Error lateral MAX [m]	0.1045	0.1044	0.1060	0.1086	0.1189

Tabla 5. Métricas controlador

Anteriormente no se tomaron valores menores a 0.25 porque el controlador comenzaba a oscilar y por ende se demoraba en completar la trayectoria. A continuación, se presentará las gráficas de los valores de **Ld 0.25, 0.4 y 0.8** para hacer una correlación y comparación, aunque aún a simple vista con las métricas de error se podría asumir que el **Ld** de **0.4** sería el más óptimo para este controlador.

Pure pursuit Ld de 0.25: Al usar un punto futuro a una distancia demasiado corta, el controlador se acopla bien en los puntos diagonales que A* lo que demuestra una respuesta rápida en curvas cerradas, más sin embargo hay una curva que cuestiona la teoría del control pure pursuit ya que es muy cerrada y el controlador no se puede acoplar a la referencia, pero aun así el seguimiento es correcto.

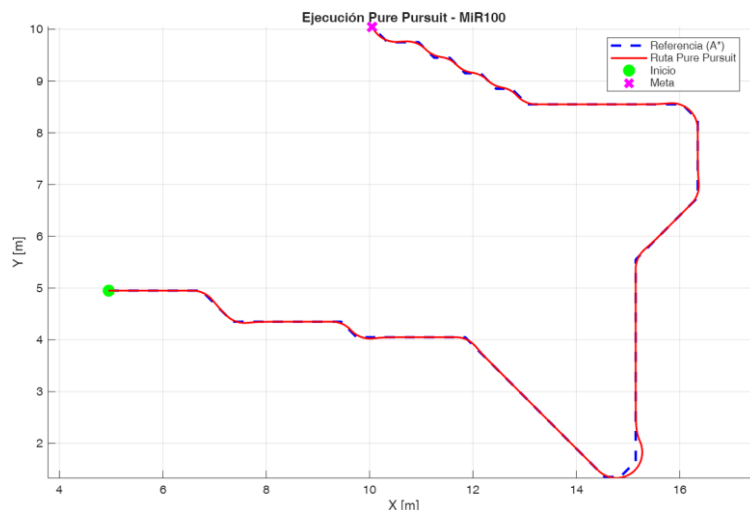


Figura 36. Control con Ld 0.25

Como se mencionó anteriormente esta oscilación que se refleja en la trayectoria puede observarse en la gráfica 31 de comparación de velocidad angular vs tiempo, como podemos observar no hay mucha oscilación, lo que genera una velocidad lineal constante exitosa para un controlador.

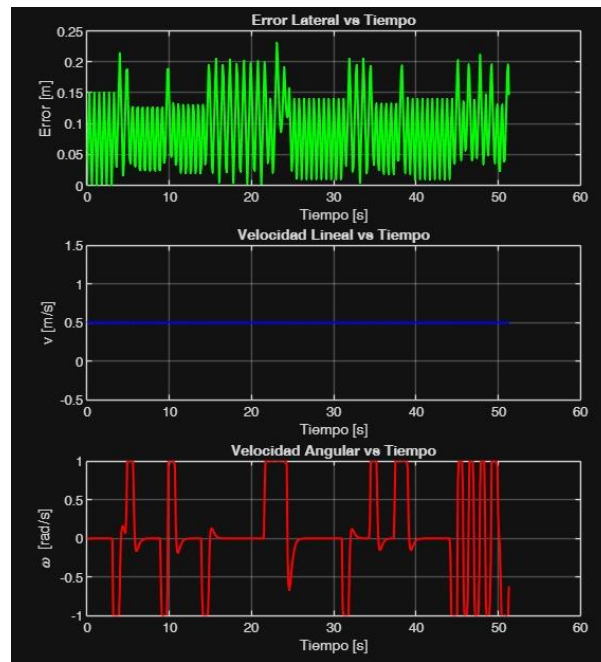


Figura 37. Evolución L_d 0.25

Pure Pursuit L_d de 0.4: Al elevar el valor del punto futuro en muy poco se ve un seguimiento de la trayectoria aún más exitoso, más sin embargo se resalta que comienza a perder seguimiento en las curvas cerradas rápidas, pero la curva más complicada si la está cumpliendo. Lo cual indica que este valor de L_d si se podría usar.

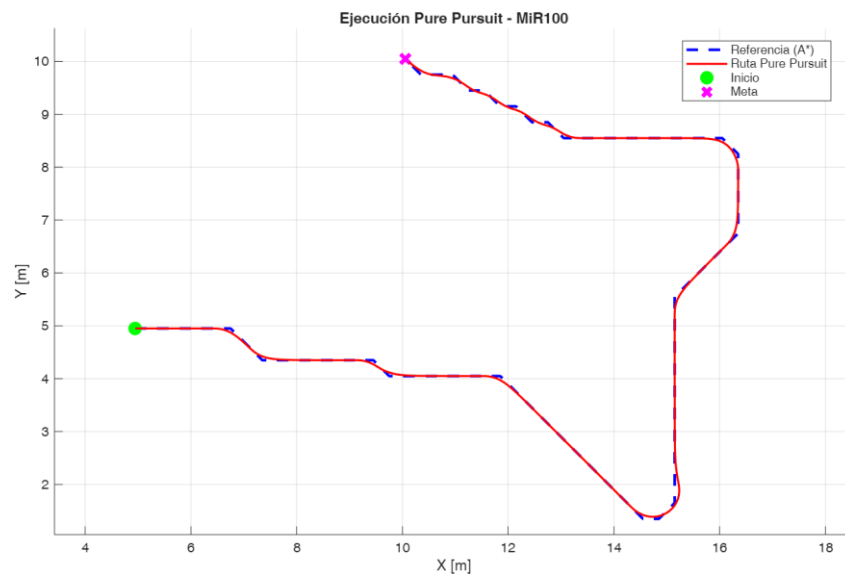


Figura 38. Control con L_d 0.4

Al igual que el anterior caso, la velocidad angular tiene la misma oscilación, con la diferencia que la intensidad en rad/s es menor, y la velocidad lineal se conserva constante.

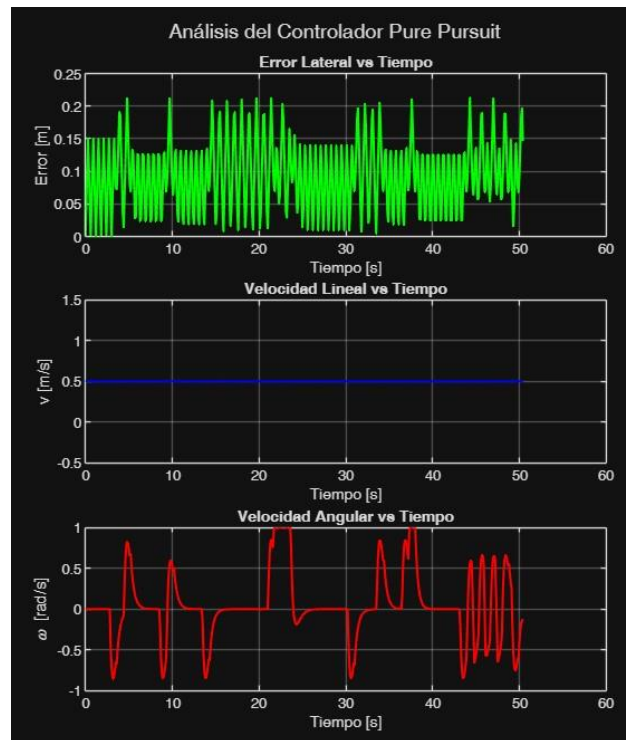


Figura 39. Evolución con L_d 0.4

Pure pursuit L_d de 0.8: En la figura 40 se puede observar el otro extremo con un L_d , que, al tener un punto futuro más lejano, las curvas cerradas han perdido precisión y el controlador va en linealidad; como consecuencia la curva más complicada la ha cerrado demasiado. Igualmente, en este caso si bien la amplitud de la velocidad angular es menor en caso, el error lateral con mayor frecuencia de a probar que este tipo de control pueda no ser el más indicado.

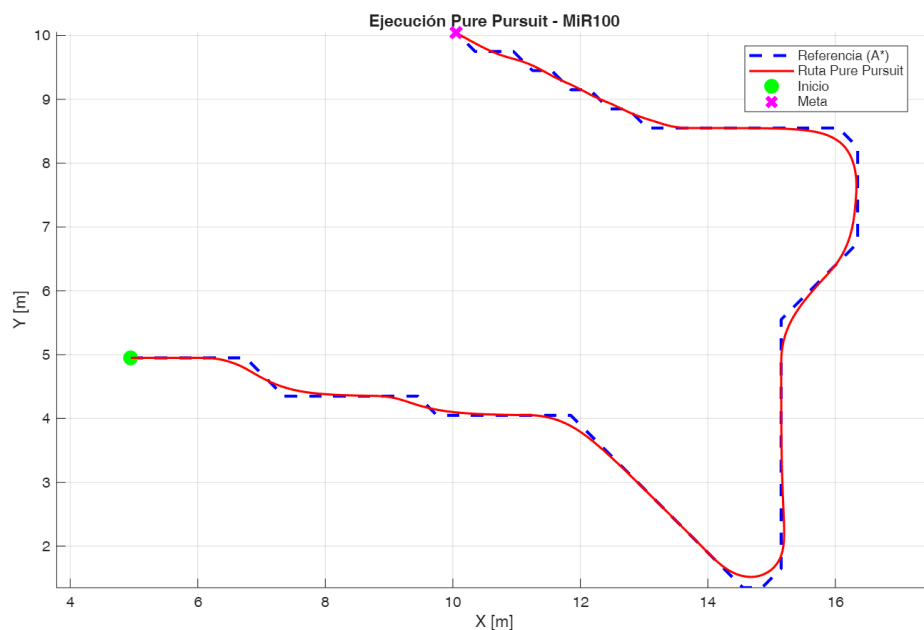


Figura 40. Control con L_d 0.8

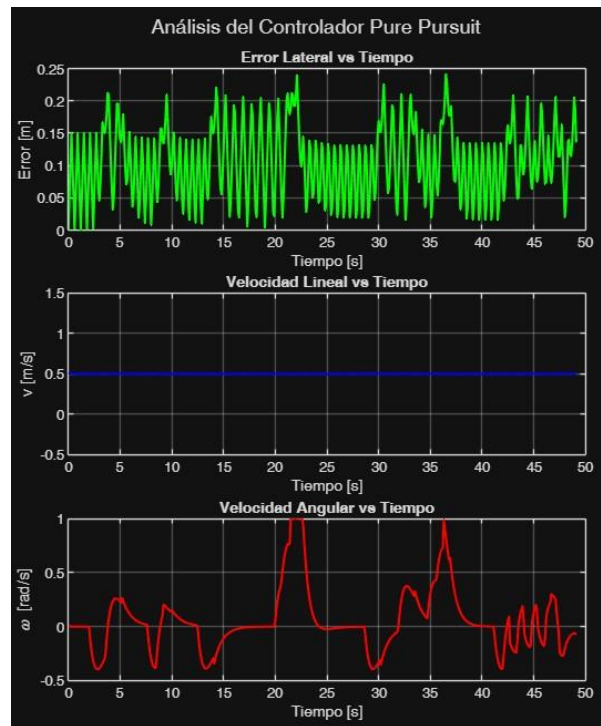


Figura 41. Evolución con L_d 0.8

Superposición trayectoria sobre el mapa: Finalmente es muy importante para realizar una adecuada comprobación, evidenciar por simulación cómo se comporta esta trayectoria sobre el mapa original, en este caso la trayectoria del controlador se voltea verticalmente para que su eje 0,0 coincida con el del mapa importado.

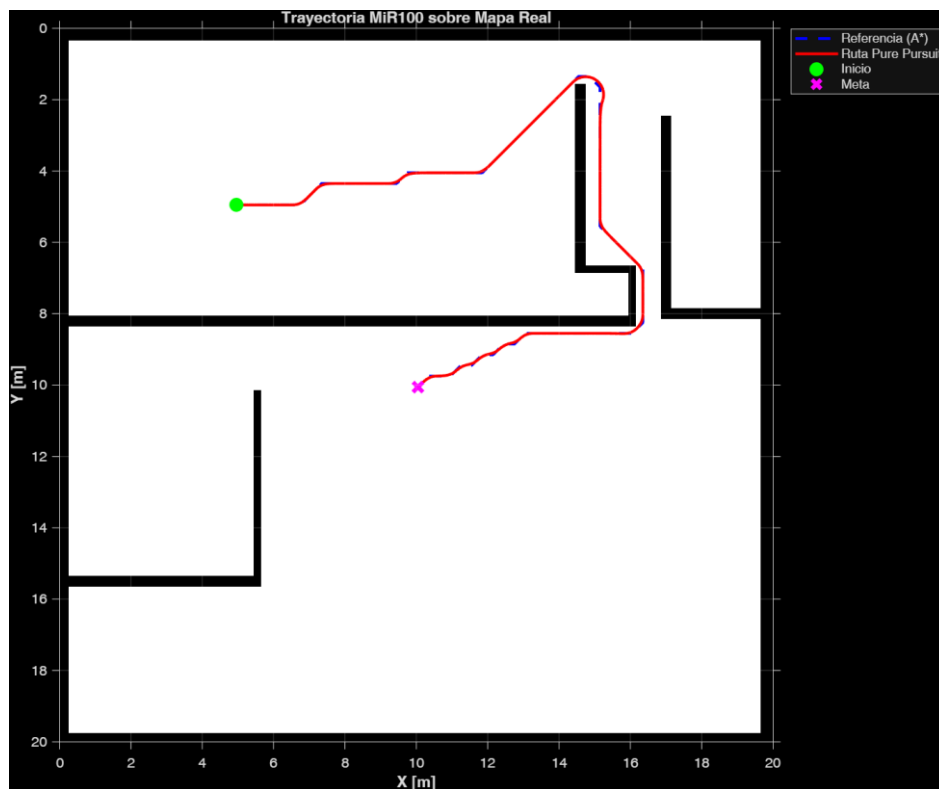


Figura 42. Mapa y trayectoria con L_d 0.25

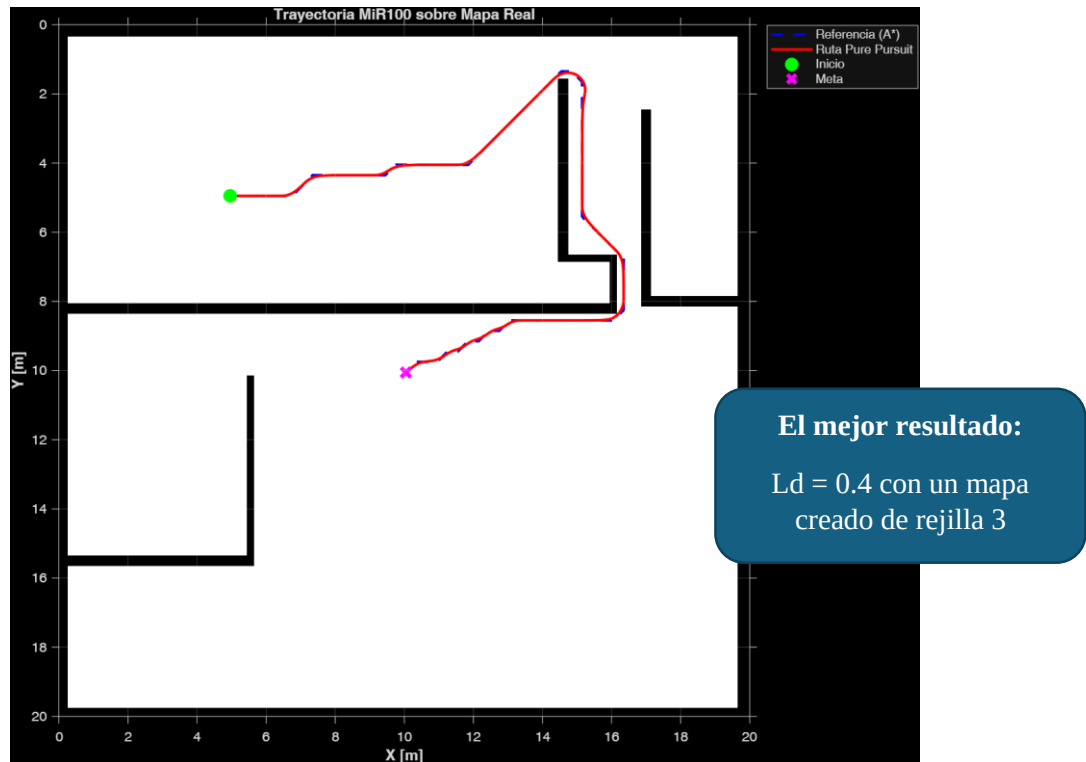


Figura 43. Mapa y trayectoria con L_d 0.4

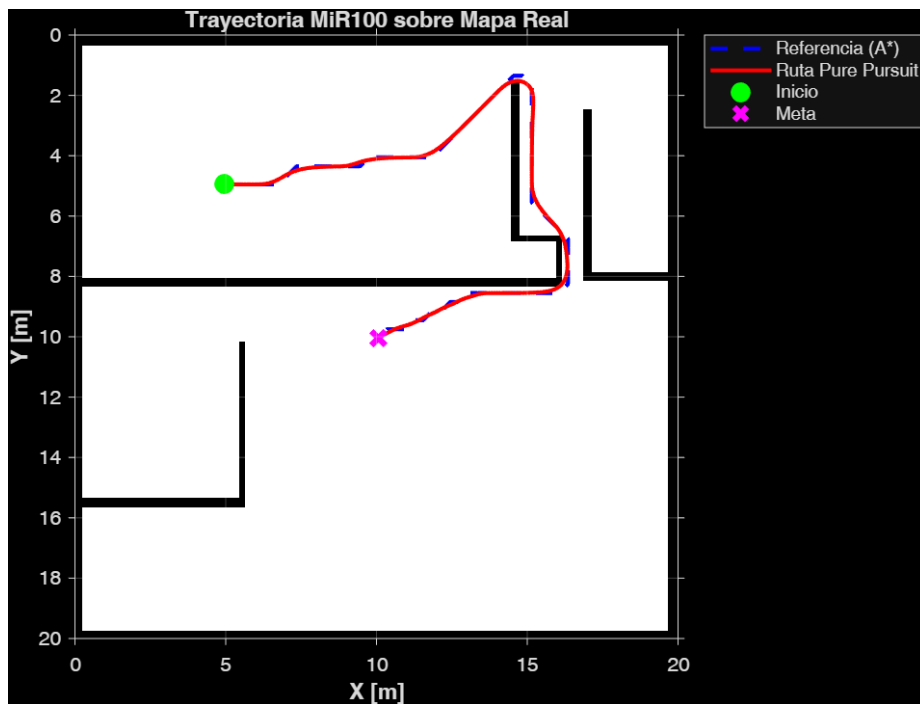


Figura 44. Mapa y trayectoria con L_d 0.8

Si bien se observa que las gráficas con valores de 0,25 y 0,3 cumplen adecuadamente, se obtará por usar finalmente la que menos error ha suministrado en RMS de acuerdo a las métricas y esta sería la gráfica del controlador Pure Purisuit con L_d de 0.4

Finalmente se realiza un análisis del controlador pure pursuit con las diferentes parametrizaciones del lookahead (Ld), comprobando que hay que evitar Ld inferiores a 0.3 o superiores a 0.6, tomando las alternativas de los Ld cercanos a 0,4.

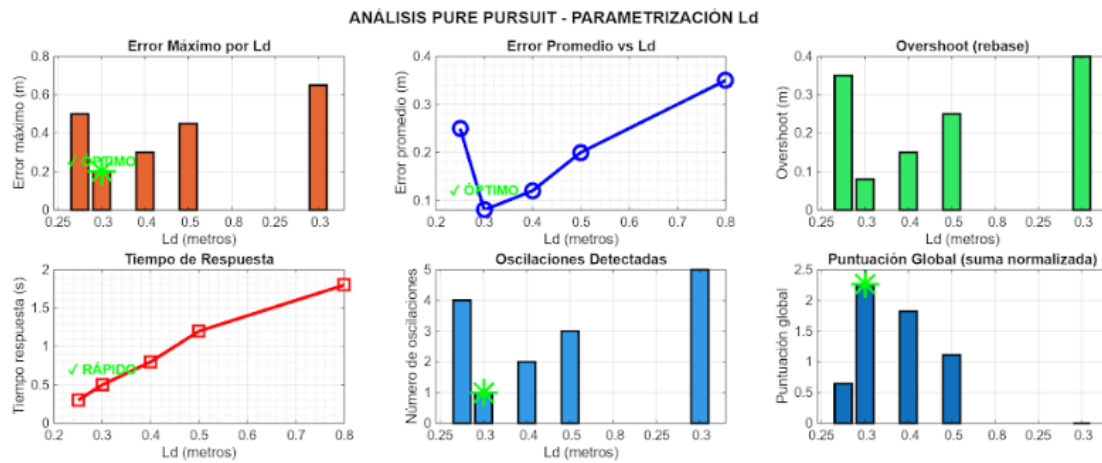


Figura 45. Comparación parametrización Ld

PRUEBAS ADICIONALES: Algo importante es que se aprovechó a utilizar los mismos parámetros cambiando el valor del tamaño de las rejillas del mapa para planificación, A continuación, se muestra cómo influye el tamaño de la rejilla del mapa en la planificación y por ende en la trayectoria final, donde claramente se observa que la trayectoria controlada pasa por encima de los obstáculos y es por eso que se tomó el valor de rejilla igual a 3 para implementar el controlador, porque fue el que menos errores daba al final de la trayectoria.

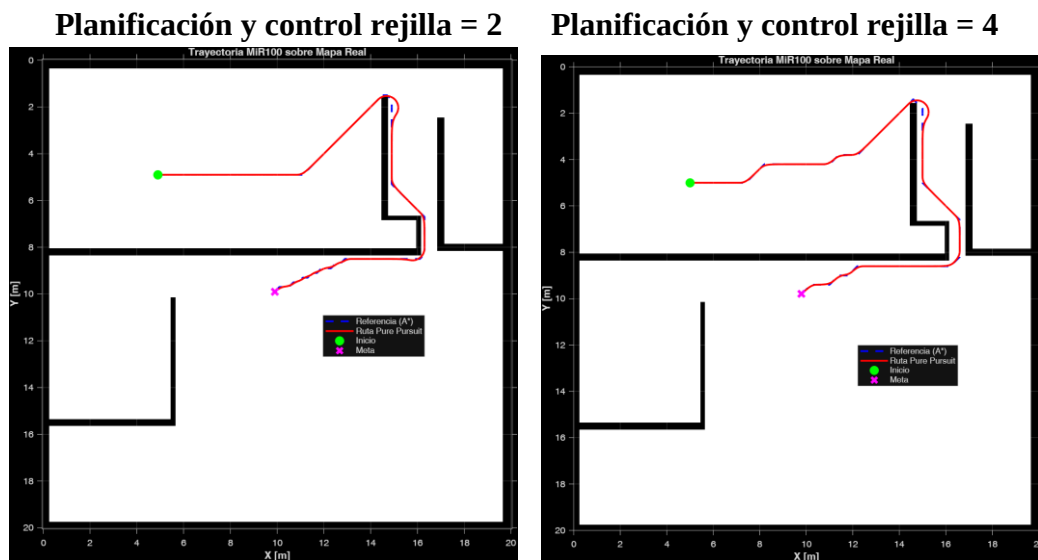


Figura 46. Comparación rejillas

Enlace repositorio gitHub: Se comparte el enlace del repositorio en github el cual es público y se puede ver las diferentes versiones, pruebas y scripts trabajados. La actualización de la parte 3 se ha agregado a la carpeta Planeación

Video explicación: <https://youtu.be/SLBZ2afxZzU>

III. CONCLUSIONES

Se resalta que para estas trayectorias a diferencia de A* no se realizó el ensanchamiento de los obstáculos, por lo tanto, al realizar los cálculos matemáticos de RRT* se puede presentar la trayectoria sobreexpuesta sobre los mismos. Por lo tanto, se recomienda ensanchar estos obstáculos o considerar la dimensión del robot, algo que A* soluciona más sencillo al hacer el mapa por medio de rejillas con tamaños ya definidos

Luego de agrupar todas las fases del proyecto, es necesario resaltar algunos apartados importantes, comenzando por los filtros de localización. Si bien tras evaluar los diferentes filtros de localización, aunque UKF mostró mejores métricas iniciales, intentamos mapear con EKF por su facilidad de programación. Sin embargo, EKF tuvo problemas para alinear correctamente obstáculos complejos en SLAM, lo que se convirtió en una frustración de día, por lo tanto se aprovechó el filtro AMCL suministrado por el ROSBAG, el cual ayudó a generar un mapa de ocupación mucho más preciso sin dejar de lado las caudidades del SLAM puro evitando depender de valores ideales impracticables como el Ground Truth o de la odometría sin filtro.

En cuanto a la planificación de trayectorias, comparando los métodos de planificación más especializados vistos en el curso basados en muestreo y rejilla, se determinó que A* es muy superior para este mapa en específico y para alguna trayectoria compleja; mientras RRT* tardaba varios segundos en recalcular rutas para evadir obstáculos dinámicos y no siempre los mismos parámetros funcionaban, A* logró encontrar nuevas rutas en apenas unos pocos milisegundos. Igualmente se resalta cómo influye también el tamaño de celdas de este mapa en la trayectoria controlada, lo que da a entender que se debe escoger con cuidado para evitar obstáculos.

Para el controlador Pure Pursuit, que se escogió por su respuesta y facilidad de ajuste. Tras sintonizar la distancia de anticipación (Lookahead Distance), se determinó que un valor de 0.4 resultó ser el más óptimo. Este valor minimizó el error lateral, permitiendo un seguimiento estable de la trayectoria sin perder precisión en las curvas más cerradas del mapa y evitando colisiones.

Finalmente, es grato observar como las fases individuales vistas en el curso complementan toda la robótica móvil.

IV. REFERENCIAS

Client challenge. (n.d.). <https://www.scribd.com/document/872329584/MiR-MiR100-User-Guide>

Hercik, R., Byrtus, R., Jaros, R., & Koziorek, J. (2022). Implementation of autonomous mobile robot in SmartFactory. *Applied Sciences*, 12(17), 8912. <https://doi.org/10.3390/app12178912>

LaValle, Steven M. (2006). *Planning algorithms*. Cambridge University Press.

MATLAB. Disponible en: <<https://matlab.mathworks.com/>>.

Mobile Industrial Robot 100 - Robotplus. (2023, 10 febrero).
Robotplus.<https://robotplus.es/mobile-industrial-robot-100/>

Siciliano, B., Sciavicco, L., Villani, L., & Oriolo, G. (2009). *Robotics: modelling, planning and control*. Springer London.

Siegwart, R., Nourbakhsh, I. R., & Scaramuzza, D. (2011). *Introduction to autonomous mobile robots*. MIT press. Thrun, S., Burgard, W., & Fox, D. (2005). *Probabilistic Robotics*. Cambridge, MA: MIT Press.

Thrun, S., Burgard, W., & Fox, D. (2005). *Probabilistic Robotics*. Cambridge, MA: MIT Press.

Ugwoke, K. C., Nnanna, N. A., & Abdullahi, S. E. Y. (2025). Simulation-based review of classical, heuristic, and metaheuristic path planning algorithms. *Scientific Reports*, 15(1), 12643.